E3

ESP32 Education Editor

ユーザーズガイド & ブロックリファレンス

Editor v0.6 / Firmware v0.1.8

対象: ESP32-WROOM-32 / 日本語版

2026年10月

目次

PART 1 ユーザーズガイド

1. ESP32 Education Editorとは
2. 必要なもの
3. 初回準備
4. 画面構成
5. ESP32へ接続する
6. 拡張カテゴリーを追加する
7. 基本的なプログラムの作り方
8. プログラムの実行
9. プロジェクトの保存と読み込み
10. ESP-NOWの利用
11. ハードウェア利用上の注意
12. 困ったとき
13. 関連文書
14. 位置づけ

PART 2 ブロックリファレンス

ESP32 接続

ESP32 GPIO

ESP32 DHT

ESP32 OLED

ESP32 ESP-NOW

ESP32 Servo

ESP32 超音波

ESP32 環境指数

データ処理

本PDFのブロック図は、現行ソースの日本語表示・既定値・カテゴリー色・ブロック種別を基にした説明用SVGです。

PART 1

ESP32 Education Editor ユーザーズガイド

1. ESP32 Education Editorとは

ESP32 Education Editorは、ESP32-WROOM-32をブラウザからブロックで操作する教育向けプログラミング環境です。ESP32側にはFirmware v0.1.8を書き込み、EditorからUSB / Web Serialで命令を送ります。

- Editor: **v0.6**
- Firmware: **v0.1.8**
- 対象ボード: **ESP32-WROOM-32 / ESP32 Dev Module**
- 公開Editor: https://davinichi.github.io/
- Firmware Installer: https://davinichi.github.io/firmware/



2. 必要なもの

- ESP32-WROOM-32
- データ通信可能なUSBケーブル
- ChromeまたはEdgeなどWeb Serial対応ブラウザ
- インターネット接続（公開Editor・Installerの読み込み時）
- 使用するセンサー、LED、OLED、サーボモーターなど
- 回路に応じた抵抗や電源

> HC-SR04のECHO出力は約5Vです。ESP32へ直接入力せず、3.3V以下へ下げてください。

3. 初回準備



3.1 Firmwareを書き込む

1. ESP32-WROOM-32をUSBでPCへ接続します。
2. Firmware Installerを開きます。
3. 「ESP32に接続して v0.1.8 を書き込む」を押します。
4. ESP32のUSBシリアルポートを選びます。
5. 画面の案内に従って書き込みます。

Arduino IDEや追加ライブラリを利用者側で準備する必要はありません。

3.2 Editorを開く

公開EditorをChromeまたはEdgeで開きます。上部の言語メニューから日本語／Englishを切り替えられます。

4. 画面構成



- **上部メニュー**: 言語切替、ファイル、新規作成、保存・読み込み、Firmware Installer、About

- **カテゴリー／ブロック**: 標準カテゴリーと追加したESP32拡張
- **スクリプト作成エリア**: ブロックを組み立てる場所
- **ステージ**: スプライトの表示と実行結果の確認
- **スプライト／背景**: スプライトや背景の追加・選択

5. ESP32へ接続する

1. 画面左下の拡張機能ボタンを押します。
2. **ESP32 接続**を追加します。
3. 「ESP32に接続」ブロックを実行します。
4. 表示された一覧からESP32のシリアルポートを選びます。
5. 必要に応じて「ESP32の状態を更新」を実行し、状態・MACアドレス・チャンネルを確認します。

同じシリアルポートをArduino IDEのシリアルモニターなどが使用している場合は、先に切断してください。

6. 拡張カテゴリーを追加する

利用目的に応じて拡張カテゴリーを追加します。

カテゴリー	主な用途	ESP32接続
ESP32 接続	接続、状態、MAC、チャンネル	必要
ESP32 GPIO	デジタル入出力、PWM	必要
ESP32 DHT	DHT11 / DHT22温湿度	必要
ESP32 OLED	SSD1306 OLED表示	必要
ESP32 ESP-NOW	ESP32間の文字列通信	必要
ESP32 Servo	サーボモーター	必要
ESP32 超音波	HC-SR04距離測定	必要
ESP32 環境指数	10種類の環境指数計算	不要
データ処理	CSV・文字列処理	不要

各ブロックの詳細はブロックリファレンスを参照してください。

7. 基本的なプログラムの作り方

7.1 LEDを点灯する例

1. ESP32 接続を追加し、ESP32へ接続します。
2. ESP32 GPIOを追加します。
3. 「GPIO 23 を 出力 に設定」を実行します。
4. 「GPIO 23 に HIGH を出力」で点灯します。
5. 「GPIO 23 に LOW を出力」で消灯します。

LEDには電流制限抵抗を使用し、配線を確認してください。

7.2 センサー値を表示する例

1. DHTセンサーを初期化します。
2. DHT温度とDHT湿度を読みます。
3. OLEDを初期化します。
4. カーソル位置と文字サイズを設定します。
5. 取得値を文字列にしてOLEDへ表示します。

8. プログラムの実行

Scratch標準のイベントブロックや制御ブロックと、ESP32拡張ブロックを組み合わせます。

- 緑の旗が押されたとき
- ずっと
- ◯秒待つ
- もし〜なら
- ◯回繰り返す
- 変数・演算

現在のEditorはLive型です。プログラム実行中はPC、ブラウザ、USB接続が必要です。

9. プロジェクトの保存と読み込み

上部の**ファイル**メニューから操作します。

- **新規**: 新しいプロジェクトを開始
- **コンピューターから読み込む**: 保存済みプロジェクトを開く
- **コンピューターに保存する**: 現在のプロジェクトをPCへ保存

日本語表示と英語表示では内部のextension ID・opcodeを共通化しているため、同じプロジェクトを両言語で開けるよう設計されています。

10. ESP-NOWの利用

ESP-NOWはESP32同士が直接通信する仕組みです。

- 学校Wi-Fiアクセスポイント: 不要
- 校内LAN: 不要
- ESP-NOW通信時のインターネット: 不要
- 送受信するESP32のチャンネル: 同じ値に設定
- MAC空欄の送信: ブロードキャスト
- MAC指定の送信: ユニキャスト

公開EditorやFirmware InstallerをWebから読み込む際はインターネット接続が必要です。

11. ハードウェア利用上の注意

- 対象はESP32-WROOM-32です。
- GPIO番号、電源電圧、入力専用ピンなどは使用するボードの仕様を確認してください。
- LEDには電流制限抵抗を使用します。
- サーボモーターの電源条件は使用する製品の仕様を確認します。
- HC-SR04のECHOは3.3V以下へ下げます。
- 配線変更時は必要に応じて電源やUSBを外します。
- 同じGPIOを複数の機能で同時使用しないようにします。

12. 困ったとき

最初に次を確認してください。

1. 対象ボードがESP32-WROOM-32か
2. Firmware v0.1.8を書き込んだか
3. Chrome / Edgeを使っているか
4. データ通信可能なUSBケーブルか
5. シリアルポートを他アプリが使用していないか
6. 使用する拡張を追加したか
7. センサーの種類、GPIO、配線、電源が正しいか

詳細はFAQ / トラブルシューティングを参照してください。

13. 関連文書

- ブロックリファレンス

- Getting Started
- FAQ / トラブルシューティング
- Firmwareガイド
- 授業・教育利用ガイド
- バックアップ・レストア手順書

14. 位置づけ

ESP32 Education EditorはScratch Foundationが公開するオープンソースのScratch Editorを基にした独立プロジェクトです。Scratch Foundationの公式製品、承認、推奨、スポンサーを意味するものではありません。

PART 2

ESP32 Education Editor ブロックリファレンス

ESP32 Education Editor v0.6でパレットに表示される49個の公開ブロックを、カテゴリー別に説明します。

> ブロック図は現行ソースの日本語表示、既定値、カテゴリー色、ブロック種別を基にした説明用SVGです。実際の表示はブラウザや画面幅によって多少異なります。

ブロック形状

- **コマンドブロック**: 動作を実行します。
- **レポーターブロック**: 数値や文字列を返します。
- **真偽値ブロック**: 真／偽を返し、条件式に使います。

目次

- ESP32 接続
- ESP32 GPIO
- ESP32 DHT
- ESP32 OLED
- ESP32 ESP-NOW
- ESP32 Servo
- ESP32 超音波
- ESP32 環境指数
- データ処理

ESP32 接続

USB / Web SerialでESP32へ接続し、接続状態、MACアドレス、Wi-Fiチャンネルを確認します。

利用条件: ESP32とFirmware v0.1.8が必要です。このカテゴリーを最初に追加します。

1. ESP32に接続

ESP32に接続

種類: コマンドブロック

機能: ブラウザのWeb Serialを使ってESP32へ接続します。

入力: なし

出力: なし

注意: 実行するとブラウザのシリアルポート選択画面が開きます。Arduino IDEのシリアルモニターなど、同じポートを使うアプリは先に閉じます。

使用例: 最初にこのブロックを実行してから、GPIOやセンサーのブロックを使います。

2. ESP32から切断

ESP32から切断

種類: コマンドブロック

機能: ESP32とのWeb Serial接続を閉じます。

入力: なし

出力: なし

注意: 別のアプリでシリアルポートを使う前や、USBケーブルを外す前に使用します。

使用例: プログラム終了時にESP32との接続を明示的に閉じます。

3. ESP32は接続済み？

ESP32は接続済み？

種類: 真偽値ブロック

機能: 現在ESP32と接続しているかを真偽値で返します。

入力: なし

出力: 接続中なら「真」、未接続なら「偽」

注意: 接続状態によって処理を分けたいときに使います。

使用例: 「もし ESP32は接続済み？ なら」の条件に使います。

4. ESP32の状態を更新

ESP32の状態を更新

種類: コマンドブロック

機能: ESP32へ状態情報の送信を要求します。

入力: なし

出力: なし

注意: 状態、MACアドレス、Wi-Fiチャンネルを最新にしたいときに実行します。

使用例: 接続後に実行し、続けて状態やMACアドレスのレポーターブロックを読みます。

5. ESP32の状態

ESP32の状態

種類: レポーターブロック

機能: ESP32から最後に受信した状態文字列を返します。

入力: なし

出力: 状態文字列。未取得時は空文字列になる場合があります。

注意: 接続後または「ESP32の状態を更新」の後に確認します。

使用例: 状態をステージ上に表示して接続確認に使います。

6. ESP32のMACアドレス

ESP32のMACアドレス

種類: レポーターブロック

機能: 接続中のESP32自身のMACアドレスを返します。

入力: なし

出力: MACアドレス文字列

注意: ESP-NOWで送信先を指定するときの確認に使えます。

使用例: 2台のESP32で互いのMACアドレスを確認します。

7. ESP32のWi-Fiチャンネル

ESP32のWi-Fiチャンネル

種類: レポーターブロック

機能: ESP32が現在使用しているWi-Fiチャンネルを返します。

入力: なし

出力: チャンネル番号

注意: ESP-NOWでは送受信するESP32同士のチャンネルをそろえます。

使用例: ESP-NOWが受信できないときの確認に使います。

ESP32 GPIO

GPIOの入出力モード、デジタル出力、デジタル入力、PWM出力を扱います。

利用条件: ESP32への接続が必要です。使用するGPIOと接続回路を確認してください。

8. GPIO [PIN] を [MODE] に設定



種類: コマンドブロック

機能: 指定したGPIOの入出力モードを設定します。

入力: PIN: GPIO番号 / MODE: 出力、入力、入力（プルアップ）、入力（プルダウン）

出力: なし

注意: デジタル出力や入力を行う前に、用途に合ったモードを設定します。

使用例: GPIO23を「出力」に設定してからLEDを点灯します。

9. GPIO [PIN] に [STATE] を出力



種類: コマンドブロック

機能: 指定GPIOへHIGHまたはLOWを出力します。

入力: PIN: GPIO番号 / STATE: HIGH または LOW

出力: なし

注意: GPIOを出力モードにしてから使用します。

使用例: GPIO23をHIGHにしてLEDを点灯し、LOWにして消灯します。

10. GPIO [PIN] のPWM出力を [PERCENT] % にする



種類: コマンドブロック

機能: 指定GPIOへ0〜100%のPWMを出力します。

入力: PIN: PWM対応メニューのGPIO / PERCENT: 0〜100

出力: なし

注意: 入力値は整数に丸められ、0〜100の範囲に制限されます。Firmware v0.1.8では5 kHz・8-bitです。

使用例: LEDの明るさを50%程度にします。

11. GPIO [PIN] のPWM出力を停止する



種類: コマンドブロック

機能: 指定GPIOのPWM出力を停止します。

入力: PIN: PWM対応メニューのGPIO

出力: なし

注意: 停止後に再びPWM出力を開始できます。

使用例: PWMによるLED調光を終了します。

12. GPIO [PIN] のデジタル入力



種類: レポーターブロック

機能: 指定GPIOのデジタル入力値を読み取ります。

入力: PIN: GPIO番号

出力: 0 または 1

注意: 読み取り要求後、最後に受信した値を返します。未取得時は0になる場合があります。

使用例: スイッチ入力が1になったらLEDを点灯します。

ESP32 DHT

DHT11 / DHT22から温度と相対湿度を取得します。

利用条件: ESP32への接続と、DHTセンサーの種類・GPIOの初期化が必要です。

13. [TYPE] を GPIO [PIN] で初期化



種類: コマンドブロック

機能: DHTセンサーの種類と接続GPIOをFirmwareへ設定します。

入力: TYPE: DHT11 または DHT22 / PIN: GPIO番号

出力: なし

注意: 温度・湿度を読む前に1回実行します。

使用例: DHT11をGPIO25へ接続した場合は、TYPEをDHT11、PINを25にします。

14. DHT 温度（℃）



種類: レポーターブロック

機能: DHTセンサーを読み取り、温度を摂氏で返します。

入力: なし

出力: 温度（℃）。読み取り失敗時は0になる場合があります。

注意: 実行時にESP32へDHT読み取りを要求します。

使用例: 温度が30℃以上なら警告を表示します。

15. DHT 湿度（%）



種類: レポーターブロック

機能: DHTセンサーを読み取り、相対湿度を返します。

入力: なし

出力: 相対湿度（%）。読み取り失敗時は0になる場合があります。

注意: 実行時にESP32へDHT読み取りを要求します。

使用例: 湿度をOLEDへ表示します。

16. DHTの状態

DHTの状態

種類: レポーターブロック

機能: DHTから最後に受信した状態文字列を返します。

入力: なし

出力: 状態文字列

注意: 初期化や読み取りが正常かを確認する補助ブロックです。

使用例: 接続確認時に状態を表示します。

ESP32 OLED

SSD1306 128×64 OLEDの初期化、文字表示、部分消去、状態確認を行います。

利用条件: ESP32への接続と、Firmwareが想定するSSD1306 OLEDの配線が必要です。

17. OLEDを初期化



種類: コマンドブロック

機能: OLEDを初期化します。

入力: なし

出力: なし

注意: 他のOLEDブロックを使う前に実行します。

使用例: 緑の旗が押されたときに最初に実行します。

18. OLED画面を消去



種類: コマンドブロック

機能: OLED画面全体を黒で消去します。

入力: なし

出力: なし

注意: 画面全体を書き直す前に使用します。

使用例: 新しい測定値を表示する前に画面を消去します。

19. OLEDの X [X] Y [Y] 幅 [W] 高さ [H] を黒で塗りつぶす



種類: コマンドブロック

機能: 指定した矩形領域だけを黒で塗りつぶします。

入力: X: 0〜127 / Y: 0〜63 / W: 幅 / H: 高さ

出力: なし

注意: 座標と大きさは128×64の画面範囲内に制限されます。

使用例: 数値表示部分だけを消して更新します。

20. OLEDカーソルを X [X] Y [Y] に設定



種類: コマンドブロック

機能: 次に表示する文字の開始位置を設定します。

入力: X: 0〜127 / Y: 0〜63

出力: なし

注意: 画面左上がX=0、Y=0です。

使用例: X=0、Y=16に設定して2行目相当へ表示します。

21. OLED文字サイズを [SIZE] に設定



種類: コマンドブロック

機能: OLEDに表示する文字サイズを設定します。

入力: SIZE: 1〜8

出力: なし

注意: 値は整数に丸められ、1〜8に制限されます。

使用例: 見出しを大きく表示するときに2を指定します。

22. OLEDに [TEXT] を表示



種類: コマンドブロック

機能: 現在のカーソル位置へ文字列を表示します。

入力: TEXT: 表示する文字列

出力: なし

注意: 改行文字は空白へ置き換えられます。

使用例: 温度と湿度を文字列にして表示します。

23. OLED表示テスト

OLED表示テスト

種類: コマンドブロック

機能: Firmwareに用意されたOLED表示テストを実行します。

入力: なし

出力: なし

注意: 配線や初期化を確認するときに使用します。

使用例: 授業前の動作確認で実行します。

24. OLEDの状態

OLEDの状態

種類: レポーターブロック

機能: OLEDから最後に受信した状態を返します。

入力: なし

出力: READY、ERROR、または空文字列

注意: 初期化の結果確認に使います。

使用例: READYでなければ配線を確認します。

25. OLED最後の応答

OLED最後の応答

種類: レポーターブロック

機能: OLED操作に対する最後の応答内容を返します。

入力: なし

出力: 最後のACK文字列

注意: 初期化、消去、カーソル設定、文字表示などの応答確認に使います。

使用例: デバッグ時に応答を表示します。

ESP32 ESP-NOW

学校Wi-Fiや校内LANを使わず、ESP32同士で文字列を直接送受信します。

利用条件: 送受信するESP32にFirmware v0.1.8が必要です。チャンネルをそろえ、必要に応じてMACアドレスを指定します。

26. ESP-NOW チャンネルを [CHANNEL] にする



種類: コマンドブロック

機能: ESP-NOWで使用するWi-Fiチャンネルを1〜13から設定します。

入力: CHANNEL: 1〜13

出力: なし

注意: 通信するESP32同士で同じチャンネルを使用します。再起動後は1へ戻ります。

使用例: 送信側と受信側を同じチャンネル6へ設定します。

27. ESP-NOW チャンネル



種類: レポーターブロック

機能: 現在のESP-NOWチャンネルを返します。

入力: なし

出力: チャンネル番号。未接続時は空文字列。

注意: 接続後のチャンネル確認に使用します。

使用例: 通信できないときに両方の値を比較します。

28. 送信先MAC [MAC] に [MESSAGE] を送信



種類: コマンドブロック

機能: ESP-NOWで文字列を送信します。

入力: MAC: 送信先MACアドレス（空欄可） / MESSAGE: 送信する文字列

出力: なし

注意: MACを空欄にするとブロードキャスト、指定するとユニキャストです。改行は空白へ置き換えられます。

使用例: 空欄のまま「こんにちは」を全端末へ送信します。

29. ESP-NOWで受信したデータ

ESP-NOWで受信したデータ

種類: レポーターブロック

機能: 最後に受信したESP-NOW本文を返します。

入力: なし

出力: 受信文字列

注意: 値を読むと「新しいデータ」フラグが消えます。

使用例: 新着判定が真のときにこの値を読みます。

30. ESP-NOWの新しいデータを受信した？

ESP-NOWの新しいデータを受信した？

種類: 真偽値ブロック

機能: 未読の新しいESP-NOWデータがあるかを返します。

入力: なし

出力: 新着があれば真、なければ偽

注意: 「受信したデータ」ブロックを読むと偽へ戻ります。

使用例: ずっとループの中で新着を監視します。

31. ESP-NOWの送信結果

ESP-NOWの送信結果

種類: レポーターブロック

機能: 最後のESP-NOW送信結果を返します。

入力: なし

出力: OK、FAIL、またはエラー文字列

注意: MACアドレス形式や相手機器の設定確認に使います。

使用例: 送信後に結果がOKか確認します。

ESP32 Servo

SG90などのサーボモーターをGPIOへ接続し、角度を指定して制御します。

利用条件: ESP32への接続が必要です。電源・配線条件は使用するサーボの仕様を確認してください。

32. サーボを GPIO [PIN] に接続



種類: コマンドブロック

機能: サーボ制御に使用するGPIOを設定します。

入力: PIN: サーボ用GPIO

出力: なし

注意: 角度を設定する前に実行します。

使用例: GPIO13へサーボを接続します。

33. サーボの角度を [ANGLE] 度にする



種類: コマンドブロック

機能: 接続済みサーボの角度を指定します。

入力: ANGLE: 0〜180度

出力: なし

注意: 値は整数に丸められ、0〜180に制限されます。

使用例: 90度へ移動して中央位置にします。

34. サーボを切断する



種類: コマンドブロック

機能: サーボ出力を停止し、GPIOから切り離します。

入力: なし

出力: なし

注意: サーボ制御を終了するときに使用します。

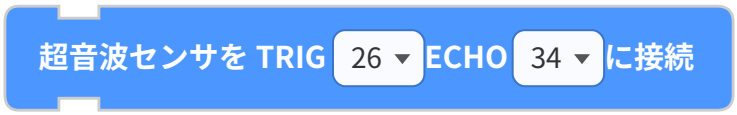
使用例: 一連の動作が終わった後に切断します。

ESP32 超音波

HC-SR04超音波センサーを使って距離をcm単位で測定します。

利用条件: ESP32への接続が必要です。HC-SR04のECHOは約5Vのため、ESP32入力を3.3V以下にしてください。

35. 超音波センサを TRIG [TRIG] ECHO [ECHO] に接続



種類: コマンドブロック

機能: HC-SR04のTRIGとECHOに使用するGPIOを設定します。

入力: TRIG: 出力用GPIO / ECHO: 入力用GPIO

出力: なし

注意: 距離を読む前に実行します。ECHOを5VのままESP32へ接続しないでください。

使用例: TRIGをGPIO26、ECHOをGPIO34に設定します。

36. 超音波センサの距離（cm）



種類: レポーターブロック

機能: HC-SR04で距離を測定してcm単位で返します。

入力: なし

出力: 距離（cm）。測定失敗時は-1。

注意: センサーの向き、対象物、配線を確認して使用します。

使用例: 距離が20cm未満なら警告音やLEDを動作させます。

ESP32 環境指数

温度と湿度から10種類の環境指数を計算します。計算はEditor内で行うためESP32接続は不要です。

利用条件: 温度と湿度の値を用意します。DHTブロックの値を入れることもできます。

選択できる環境指数

表示	内部値	概要
不快指数 DI	`DI`	体感の不快さを示す指数
Heat Index	`HEAT_INDEX`	温度と湿度から推定する暑さ指数（°C）
Humidex	`HUMIDEX`	温度と湿度から求める蒸し暑さの指数
露点温度 Td	`DEW_POINT`	水蒸気が凝結し始める温度（°C）
絶対湿度 AH	`ABS_HUM`	空気1m³あたりの水蒸気量（g/m³）
湿球温度 Tw	`WET_BULB`	湿球温度の近似値（°C）
VPD（飽差）	`VPD`	飽和水蒸気圧と実際の水蒸気圧の差（kPa）
水蒸気圧 VP	`VP`	実際の水蒸気圧（kPa）
THI（温湿度指数）	`THI`	温度と露点を使う温湿度指数
推定WBGT（簡易）	`WBGT`	湿球温度と気温から求める簡易推定値（°C）

37. 温度 [TEMP] °C 湿度 [HUM] % から [INDEX] を計算

温度 25 °C 湿度 60 % から 推定WBGT（簡易） を計算

種類: レポーターブロック

機能: 温度・湿度から選択した環境指数を計算します。

入力: TEMP: 温度（°C） / HUM: 相対湿度（%） / INDEX: 指数の種類

出力: 選択した指数の計算値

注意: 湿度は0〜100%に制限されます。結果の単位は選択した指数によって異なります。

使用例: DHT温度とDHT湿度から推定WBGTを計算します。

データ処理

CSVの分解、数値表示、文字列の切り出し・結合を行います。ESP32接続は不要です。

利用条件: Editor内で処理するため、ESP32やFirmwareは不要です。

38. CSVデータ [CSV] を分ける



種類: コマンドブロック

機能: 1行のCSV文字列をカンマで分解し、結果を内部に保存します。

入力: CSV: 分解する文字列

出力: なし

注意: 引用符で囲まれたカンマを考慮します。引用符が閉じていない場合は失敗します。

使用例: 「25.6,60,28.3」を3項目に分けます。

39. CSVの [INDEX] 番目の文字



種類: レポーターブロック

機能: 最後に分解したCSVの指定項目を文字列で返します。

入力: INDEX: 1から始まる項目番号

出力: 文字列。範囲外は空文字列。

注意: 先に「CSVデータを分ける」を実行します。

使用例: 2番目の「60」を文字列として取り出します。

40. CSVの [INDEX] 番目の数値



種類: レポーターブロック

機能: CSVの指定項目を数値へ変換して返します。

入力: INDEX: 1から始まる項目番号

出力: 数値。変換できない場合は0。

注意: 数値計算に使いたいときに使用します。

使用例: 1番目の「25.6」を数値として取り出します。

41. CSVの項目数

CSVの項目数

種類: レポーターブロック

機能: 最後に分解したCSVの項目数を返します。

入力: なし

出力: 項目数

注意: 分解前は0です。

使用例: 受信データに期待する項目数があるか確認します。

42. CSVの分解は成功した？

CSVの分解は成功した？

種類: 真偽値ブロック

機能: 最後のCSV分解が成功したかを返します。

入力: なし

出力: 成功なら真、失敗なら偽

注意: 引用符の対応が不正なCSVでは偽になります。

使用例: 分解成功時だけ各項目を利用します。

43. 数値 [VALUE] を小数点以下 [DIGITS] 桁で表示

数値 25.376 を小数点以下 2 桁で表示

種類: レポーターブロック

機能: 数値を指定した小数桁数の文字列へ変換します。

入力: VALUE: 数値 / DIGITS: 0〜10

出力: 整形済み文字列。値が数値でない場合は空文字列。

注意: DIGITSは整数化され、0〜10に制限されます。

使用例: 25.376を2桁にして「25.38」を返します。

44. 文字列 [TEXT] の左から [COUNT] 文字を取り出す



種類: レポーターブロック

機能: 文字列の先頭から指定文字数を取り出します。

入力: TEXT: 対象文字列 / COUNT: 文字数

出力: 切り出した文字列

注意: 日本語を含む文字列も文字単位で扱います。

使用例: 「ABC富山県」から「ABC」を返します。

45. 文字列 [TEXT] の右から [COUNT] 文字を取り出す



種類: レポーターブロック

機能: 文字列の末尾から指定文字数を取り出します。

入力: TEXT: 対象文字列 / COUNT: 文字数

出力: 切り出した文字列

注意: COUNTが0なら空文字列です。

使用例: 「ABC富山県」から「富山県」を返します。

46. 文字列 [TEXT] の文字数



種類: レポーターブロック

機能: 文字列の文字数を返します。

入力: TEXT: 対象文字列

出力: 文字数

注意: Unicodeコードポイント単位で数えます。

使用例: 「ABC富山県」は6を返します。

47. 文字列 [TEXT] の左から [START] 文字目以降を取り出す



種類: レポーターブロック

機能: 左から指定位置以降の文字列を返します。

入力: TEXT: 対象文字列 / START: 1から始まる開始位置

出力: 切り出した文字列

注意: STARTは1以上として扱われます。

使用例: 「2026/08/01」の6文字目以降「08/01」を返します。

48. 文字列 [TEXT] の左から [START] 文字目から [COUNT] 文字を取り出す



種類: レポーターブロック

機能: 指定位置から指定文字数を取り出します。

入力: TEXT: 対象文字列 / START: 1から始まる開始位置 / COUNT: 文字数

出力: 切り出した文字列

注意: COUNTは0以上として扱われます。

使用例: 「ABCDEFG」の3文字目から2文字「CD」を返します。

49. 文字列 [A] と [B] を [SEP] でつなぐ



種類: レポーターブロック

機能: 2つの値を指定した区切り文字で連結します。

入力: A: 前半 / B: 後半 / SEP: 区切り文字

出力: 連結した文字列

注意: 数値も文字列へ変換して連結します。

使用例: 25.6と60をカンマでつなぎ「25.6,60」を返します。

非表示の互換ブロックについて

既存プロジェクトとの互換性維持のため、`WBGT [WBGT] の警戒レベル`ブロックが内部に残っていますが、新規のブロックパレットには表示されません。本リファレンスの49公開ブロックには含めていません。

関連文書

- ユーザーズガイド
- Getting Started
- FAQ / トラブルシューティング
- Firmwareガイド